

Documento de Procedencia de Datos

Sistema de Predicción Temprana de HABs

1 Fuentes de Datos

1.1 Sentinel-2 (ESA Copernicus)

Agente:	ESA Copernicus Data Space
Producto:	S2MSI2A (Bottom-of-Atmosphere, BOA)
Bandas:	B2 (493 nm), B3 (560 nm), B4 (665 nm), B5 (705 nm), B8 (842 nm)
Resolución espacial:	10–20 m (remuestreadas a 20 m)
Resolución temporal:	5 días
Proyección:	UTM zona 17N (EPSG:32617)
Formato:	GeoTIFF, 16-bit sin signo
Periodo:	2023 a 2026
Regiones:	Okeechobee, TampaBay, Golfo de Fonseca, Lago de Yojoa, Embalse El Cajón
Directorio base:	imagenes/{Region}/{Año}/
Preprocesamiento: Las bandas se normalizan dividiendo por 10000 cuando el valor máximo supera 1.5. A partir de las 5 bandas se calculan 6 índices espectrales: NDCI, NDVI, FAI, B5/B4, B3/B2, B8/B3.	

1.2 ERA5-Land (Copernicus Climate Data Store)

Agente:	ECMWF / Copernicus CDS
API:	CDS API v3 (<code>cdsapi</code> Python)
Dataset:	<code>reanalysis-era5-land</code>
Resolución espacial:	$0.1^\circ \times 0.1^\circ$ (~ 9 km)
Resolución temporal:	4 horaria (00, 06, 12, 18 UTC)
Extensión:	Lat 31.25°N a 14.5°N , Lon 88.5°W a 79.75°W
Variables instantáneas:	t2m, u10, v10, sp
Variables acumuladas:	ssrd, tp
Periodo:	2023-01 a 2026-05
Archivos:	211 NetCDF, 219 MB total
Directorio:	<code>era5_temp_nc/</code>

Extracción: Se extrae el valor a las 12:00 UTC (índice 2 del bloque 4-horario) para cada muestra. Para el análisis de predicción temprana se extraen además valores en t-1, t-3, t-5 y t-7 días desde la misma cuadrícula ERA5.

1.3 MODIS Aqua CHL (NASA Ocean Color)

Agente:	NASA Ocean Color Web (<code>oceancolor.gsfc.nasa.gov</code>)
Producto:	MODIS Aqua L3m CHL mensual, 4 km
Algoritmo:	OC3M/OCI combinado (<code>chlora</code>)
Periodo:	2023-01 a 2026-05
Regiones:	Golfo de Fonseca, Lago de Yojoa, Embalse El Cajón
Formato:	NetCDF
Cobertura válida:	Golfo de Fonseca: 35/41 meses con CHL > 0 Lago de Yojoa: 6/41 meses con CHL > 0 El Cajón: 0/41 meses (embalse angosto, no resuelto por 4 km)
Nota:	2026 no produjo valores válidos en ninguna estación hondureña

1.4 WQP Chlorophyll (EPA/USGS)

Agente:	Water Quality Portal (www.waterqualitydata.us)
API:	REST (<code>dataRetrieval R / wqptoolbox</code>)
Parámetro:	Chlorophyll a (00665, 70960)
Unidades:	ug/L y mg/m ³
Organizaciones:	STF_WQX (South Florida Water Management), UWFCEDB (Univ. West Florida)
Periodo:	2023 a 2026
Registro válido:	59 muestras en 2025, 7 en 2026 (Panhandle de Florida)
Archivo:	<code>datasets/florida/florida_chlorophyll-_wqp_2025_2026.csv</code>
Nota:	Las muestras de 2026 (Panhandle) no se emparejaron con Sentinel-2 por falta de imágenes en esa zona

2 Scripts y Pipeline

Todos los scripts se ejecutaron en Windows 11 con PowerShell 5.1 y Python 3.13, ocupando GPU NVIDIA CUDA para el entrenamiento de redes neuronales.

2.1 `calibrar_chl_s2.py`

Propósito: Entrenar modelo RandomForest para estimar clorofila desde Sentinel-2. Usa 193 muestras reales con clorofila in-situ.

Código:

```
1 F_CHL = ["B2","B3","B4","B5","B8","NDCI","NDVI","FAI",
2         "B5_B4_ratio","B3_B2_ratio","B8_B3_ratio"]
3 chl_log = np.log10(np.maximum(chl, 0.1))
4
5 rf_chl = RandomForestRegressor(
6     n_estimators=300, max_depth=10,
7     min_samples_leaf=3, random_state=RND, n_jobs=-1
8 )
9 rf_chl.fit(X, chl_log)
10
11 pred_chl = 10 ** rf_chl.predict(X)
12 pickle.dump({"model": rf_chl, "features": F_CHL}, f)
```

Ejecución:

```
1 python scripts/calibrar_chl_s2.py
```

Métricas obtenidas:

- Pearson r: 0.85 ($p < 0.001$)
- Spearman ρ : 0.91 ($p < 0.001$)
- MAE: 6.70 ug/L

Output: modelo_final_2023/rf_chl_estimator.pkl

2.2 apply_chl_2026.py

Propósito: Aplicar el estimador de clorofila a imágenes Sentinel-2 de 2026 y extraer ERA5. Genera 1,342 muestras sintéticas.

Código (carga ERA5 y predicción):

```
1 def load_era5_month(y, m):
2     vt = ds_i.valid_time.values
3     mask = np.array([t.hour == 12
4                       for t in pd.DatetimeIndex(vt)])
5     t2m = ds_i.t2m.values[mask]
6     # ... same for sp, u10, v10, ssrd, tp
7     wspd = np.sqrt(u10**2 + v10**2)
8     return {"t2m": t2m, "sp": sp, "wspd": wspd, ...}
9
10 def get_era5_bulk(fecha, lat, lon, cache):
11     day = fecha.day - 1
12     ilat = np.argmin(np.abs(d["lats"] - lat))
13     ilon = np.argmin(np.abs(d["lons"] - lon))
14     return {"temp_air_2m": float(d["t2m"][day,ilat,ilon]),
15           "solar_radiation": float(d["ssrd"][day,ilat,ilon]),
16           "precipitation": float(d["tp"][day,ilat,ilon]),
17           "wind_speed_10m": float(d["wspd"][day,ilat,ilon]),
18           ...}
```

Código (submuestreo de píxeles):

```
1 step = max(1, int(np.sqrt(H * W / 5000)))
2 b_sub = b[:, ::step, ::step]
3 bf = b_sub.reshape(5, -1).T
4 water = bf.sum(1) > 0
5 idxs = np.random.choice(water.sum(), SAMPLES_PER_TILE, replace=False)
6
7 for idx in idxs:
8     utm_x, utm_y = tr * (col + step//2, row + step//2)
9     # UTM -> WGS84
```

```

10     lon, lat = rio_transform(
11         crs_src, {"init": "EPSG:4326"}, [utm_x], [utm_y])
12     chl_est = float(10 ** rf_chl.predict(
13         feats.reshape(1, -1))[0])
14     era = get_era5_bulk(fecha, lat, lon, era_cache)

```

Ejecución:

```
1 python scripts/apply_chl_2026.py
```

Output: datasets/samples_2026_s2_chl.csv (1,342 muestras)

2.3 lead_time_era5.py

Propósito: Extraer ERA5 de t-1, t-3, t-5, t-7 días para cada muestra, demostrando predicción temprana.

Código:

```

1 LEADS = [("t0", 0), ("t-1", -1), ("t-3", -3),
2         ("t-5", -5), ("t-7", -7)]
3 ERA_VARS = ["temp_air_2m", "solar_radiation",
4             "precipitation", "wind_speed_10m",
5             "wind_u_10m", "wind_v_10m",
6             "surface_pressure"]
7
8 for lead_name, lead_days in LEADS:
9     lead_fecha = fecha + pd.Timedelta(days=lead_days)
10    era = get_era5_at(lead_fecha, lat, lon)
11    for v in ERA_VARS:
12        entry[f"{v}_{lead_name}"] = era[v]

```

Ejecución:

```
1 python scripts/lead_time_era5.py
```

Output: datasets/lead_time_era5.csv (ERA5 multi-lead para 1,342 muestras)

2.4 clean_concat.py

Propósito: Unir datasets históricos (2023–2025) con las nuevas muestras de 2026 en un único CSV.

Código:

```

1 original = full_ds[(full_ds["fecha"].dt.year >= 2023) &
2                   (full_ds["fecha"].dt.year <= 2025) &
3                   full_ds["temp_air_2m"].notna()].copy()
4 s26 = pd.read_csv("datasets/samples_2026_s2_chl.csv")
5 s26 = s26[s26["temp_air_2m"].notna()].copy()

```

```

6
7 full = pd.concat([original_out, s26_out], ignore_index=True)
8 full = full.sort_values("fecha").reset_index(drop=True)
9 full.to_csv("datasets/dataset_entrenamiento_completo_2023_2026.csv",
  ↪ index=False)

```

Ejecución:

```

1 python scripts/clean_concat.py

```

Output: datasets/dataset_entrenamiento_completo_2023_2026.csv

3 Código de Modelos (Notebook)

El notebook `Modelo_tesis.ipynb` contiene todo el pipeline de entrenamiento, evaluación y generación de mapas.

3.1 HABNet1Dv2 (Arquitectura)

```

1 class HABNet1Dv2(torch.nn.Module):
2     def __init__(self, nf, h=128, nr=2, d=0.2):
3         super().__init__()
4         self.input_proj = nn.Sequential(
5             nn.Linear(nf,h), nn.BatchNorm1d(h),
6             nn.GELU(), nn.Dropout(d))
7         self.res_blocks = nn.ModuleList([
8             nn.Sequential(nn.Linear(h,h), nn.BatchNorm1d(h),
9                 nn.GELU(), nn.Dropout(d),
10                 nn.Linear(h,h), nn.BatchNorm1d(h))
11             for _ in range(nr)])
12         self.se = nn.Sequential(
13             nn.Linear(h,max(h//4,8)), nn.ReLU(),
14             nn.Linear(max(h//4,8),h), nn.Sigmoid())
15         self.classifier = nn.Sequential(
16             nn.Linear(h,h//2), nn.GELU(),
17             nn.Dropout(d*.5), nn.Linear(h//2,1))
18     def forward(self, x):
19         x = self.input_proj(x)
20         for b in self.res_blocks:
21             x = x + b(x) # residual connection
22         return self.classifier(x * self.se(x))

```

3.2 Entrenamiento (5-Fold CV + Hold-Out)

```

1 skf = StratifiedKFold(5, shuffle=True, random_state=RND)
2 for f, (ti, vi) in enumerate(skf.split(X, y)):
3     dl_tr = DataLoader(TensorDataset(
4         tensor(X_tr_s), tensor(y_tr).unsqueeze(1)), 32, True)
5     dl_te = DataLoader(TensorDataset(
6         tensor(X_te_s), tensor(y_te).unsqueeze(1)), 32, False)
7     m = HABNet1Dv2(len(F)).to(DEVICE)
8     o = optim.AdamW(m.parameters(), lr=1e-3, weight_decay=1e-4)
9     s = CosineAnnealingLR(o, 100, 1e-5)
10    c = FocalLoss(a=.25, g=2, pw=tensor([pw]).to(DEVICE))
11    for ep in range(200):
12        for xb, yb in dl_tr:
13            o.zero_grad()
14            c(m(xb.to(DEVICE)), yb.to(DEVICE)).backward()
15            clip_grad_norm_(m.parameters(), 1)
16            o.step()
17            # early stopping si no mejora en 25 epochs
18            if va > ba: ba = va; es = 0
19            else: es += 1
20            if es >= 25: break

```

3.3 XGBoost (Configuración)

```

1 pw_xgb = int((y_tr == 0).sum()) / max(int(y_tr.sum()), 1)
2 xgb_m = xgb.XGBClassifier(
3     n_estimators=300, max_depth=6, learning_rate=.05,
4     subsample=.8, colsample_bytree=.8,
5     scale_pos_weight=pw_xgb, random_state=RND,
6     device='cuda', tree_method='hist', verbosity=0)
7 xgb_m.fit(X_tr_s, y_tr)
8 xgb_m.save_model("modelo_final_2023/xgb_model.json")

```

3.4 Generación de Mapas (proc_tif)

```

1 def proc_tif(path, dom, st=20):
2     with rasterio.open(path) as s:
3         b = s.read().astype('float32') / 10000.
4         pr, tr = s.profile, s.transform
5         if st > 1: b = b[:, ::st, ::st]
6         bf = b.reshape(5, -1).T
7         v = bf.sum(1) > 0 # solo pixeles con agua
8         Xs = feats({sb: bv[:,i] for i,sb in enumerate(S2)})
9         if dom != 'Florida': Xs += shift # domain adaptation
10        Xf = np.column_stack([Xs, np.full((len(Xs),len(ERA_M)),ERA_M)])

```



```

11     p = xgb_m.predict_proba(sc.transform(Xf.astype('float64')))[:,1]
12     pm = np.zeros((H, W), 'float32')
13     pm.flat[v] = p
14     return pm, pr, tr

```

Domain Adaptation: El archivo `yjoa_s2_matched.csv` contiene firmas espectrales de muestras en Lago de Yojoa. Se calcula `spectral_shift.npy` como la diferencia entre la media espectral de Florida y la media de Yojoa, y se suma a las bandas de Honduras durante la generación de mapas. Adicionalmente existe `spectral_adaptation.pkl` (z-score), generado en experimentos previos. El método usado en el pipeline final es el desplazamiento aditivo (`spectral_shift.npy`).

4 Validaciones

4.1 Validación Cruzada (5-Fold Stratified)

Fold	AUC
1	0.9813
2	0.9844
3	0.9800
4	0.9795
5	0.9811
Media	0.9813 ± 0.002

4.2 Hold-Out (80/20)

Modelo	AUC	F1	Accuracy
HABNet1Dv2	0.9861	0.8871	0.9419
XGBoost	0.9948	0.9498	0.9730

4.3 Correlación con Clorofila in-situ (n = 2,424)

HABNet1Dv2: $r = 0.7314$, $\rho = 0.6247$

XGBoost: $r = 0.7471$, $\rho = 0.7709$

CHL HAB: 22.8 $\mu\text{g/L}$

CHL No-HAB: 5.4 $\mu\text{g/L}$

4.4 Estabilidad Temporal (AUC por año)

Año	Muestras	HAB	XGBoost AUC
2023	118	51	0.9918
2024	53	2	1.0000
2025	1194	306	1.0000
2026	1044	281	0.9985

4.5 Predicción Temprana (Lead-Time)

Evaluación del modelo XGBoost entrenado, substituyendo ERA5 real por ERA5 de días anteriores. Las bandas espectrales S2 se mantienen fijas.

Lead	n	AUC HABNet	AUC XGBoost	Pearson XGB
t0 (real)	1342	0.9385	0.9747	0.7960
t-1	1342	0.9370	0.9737	0.7896
t-3	1342	0.9377	0.9732	0.7976
t-5	1342	0.9351	0.9747	0.7870
t-7	1342	0.9401	0.9754	0.7974

Table 1: El AUC de XGBoost no se degrada al usar ERA5 de hasta 7 días antes. Esto valida que el sistema puede operar con pronóstico climático en lugar de ERA5 real.

4.6 Perturbación ERA5: real vs promedio

Real: HABNet AUC = 0.9956, XGB AUC = 0.9993

Promedio: HABNet AUC = 0.4824, XGB AUC = 0.9922

Cambio medio HABNet: 0.3893

Cambio medio XGBoost: 0.0447

El XGBoost es robusto al uso de ERA5 promedio, mientras que HABNet1Dv2 colapsa. Por esta razón, XGBoost es el modelo recomendado para la generación de mapas.

5 Dataset Final

Muestras totales	2,424 (645 HAB, 26.6%)
Eventos HAB	645 (26.6%)
Periodo	2023-01 a 2026-05
Variables espectrales	11 (5 bandas + 6 índices)
Variables climáticas	7 (ERA5)
Origenes	golfo_fonseca (1,422), florida (726), lago_de_yojoa (140), cajón (136)
Archivo	datasets/dataset_entrenamiento_completo_2023_2026.csv

6 Estructura de Archivos

```

1 C:\Users\JC\Desktop\Tesis\
2 +-- Modelo_tesis.ipynb
3 +-- datasets/
4 |   +-- dataset_entrenamiento_completo_2023_2026.csv
5 |   +-- lead_time_era5.csv
6 |   +-- samples_2026_s2_ch1.csv
7 |   +-- honduras_modis_ch1_monthly.csv
8 |   +-- florida/
9 |       +-- ..._wqp_2025_2026.csv
10 |       +-- florida_s2_matched.csv
11 +-- imagenes/
12 |   +-- {0keechobee,TampaBay,Cajon,
13 |       Golfo_Fonseca,Lago de Yojoa}/
14 |   +-- {2023,2024,2025,2026}/*.tif
15 +-- era5_temp_nc/    (211 archivos)
16 +-- modelo_final_2023/
17 |   +-- best_model.pth          (357 KB) HABNet final
18 |   +-- habnet1d_v2_temporal.pt (360 KB) HABNet temp
19 |   +-- habnet1d_v2_final.pt   (357 KB) HABNet final alt
20 |   +-- xgb_model.json         (815 KB) XGBoost final
21 |   +-- xgboost_final.pkl      (278 KB) XGBoost final alt
22 |   +-- xgb_temporal.pkl       (443 KB) XGBoost temporal
23 |   +-- scaler.pkl             StandardScaler
24 |   +-- scaler_final.pkl       StandardScaler alt
25 |   +-- scaler_temporal.pkl    StandardScaler temp
26 |   +-- rf_ch1_estimator.pkl    (1.5 MB) RF CHL
27 |   +-- spectral_shift.npy      Domain adaptation shift
28 |   +-- spectral_adaptation.pkl Domain adaptation z-score

```

```

29 |   +-- summary_final.json           Metrics final
30 |   +-- summary_temporal.json       Metrics temporal
31 |   +-- roc_final.png, roc_temporal.png
32 +-- mapas_finales/
33 |   +-- *_hab_final.tif           (867 mapas)
34 +-- scripts/
35 |   +-- calibrar_chl_s2.py         # RF CHL estimator
36 |   +-- apply_chl_2026.py         # 2026 sample generation
37 |   +-- lead_time_era5.py         # Multi-lead ERA5
38 |   +-- clean_concat.py           # Dataset consolidation
39 |   +-- concat_2026.py            # Alt. concatenation script

```

Resumen de Ejecución

Todos los scripts se ejecutaron en el siguiente orden:

1. calibrar_chl_s2.py (RF CHL, Pearson $r=0.85$)
2. apply_chl_2026.py (1,342 muestras 2026)
3. lead_time_era5.py (ERA5 $t-1$ a $t-7$)
4. clean_concat.py (dataset final 2,424 muestras)
5. Modelo_tesis.ipynb (entrenamiento, mapas, predicción temprana)